

Predicting High Movie Ratings

Kueski Data Science Challenge

Angel Gutierrez

https://github.com/4ngelGtz/movie_rating_predictor

Abstract

Can a user’s past preferences and a movie’s reception predict a rating of at least four stars? A 30-feature XGBoost model combines historical rating behavior, recent audience response, and MovieLens Genome content affinity. On 846,774 later rating events, it achieves Average Precision of 0.8111 and ROC-AUC of 0.8213. Personalized taste alignment is the strongest individual SHAP signal, followed by the user’s rating style. Behavioral features use only earlier events; historical availability of the frozen catalog and Genome metadata remains an explicit limitation. The model provides a strong offline baseline, with predicted probabilities reasonably aligned with observed high-rating rates, even without additional calibration.

1 Problem, data, and temporal setup

The task is to estimate whether a user will give a movie at least four stars, immediately before the rating is observed. This distinguishes likely positive responses among recorded rating events. It does not model whether someone will watch or rate a movie, or evaluate recommendations for arbitrary unseen pairs. For user u , movie m , time t , and observed rating r ,

$$y = \mathbf{1}\{r \geq 4.0\}, \quad \hat{p}(u, m, t) \approx P(y = 1 \mid \text{available information at } t).$$

1.1 Data and evaluation periods

MovieLens 20M supplies 20,000,263 ratings from 138,493 users on 26,744 rated movies, from 9 January 1995 through 31 March 2015. Ratings range from 0.5 to 5.0 in half-star increments. Table 1 shows how the four sources used by the model fit together.

Dataset	Grain	Main fields	Role
<code>ratings</code>	One user–movie rating event	<code>userId</code> , <code>movieId</code> , <code>rating</code> , <code>timestamp</code>	Target and strict-prior behavioral history
<code>movies</code>	One movie	<code>movieId</code> , <code>title</code> , <code>genres</code>	Catalog context; release year is parsed from the title
<code>genome_scores</code>	One movie–tag relevance observation	<code>movieId</code> , <code>tagId</code> , <code>relevance</code>	Movie-content vectors; relevance is metadata, not a user preference
<code>genome_tags</code>	One Genome tag	<code>tagId</code> , human-readable label	Interprets vector dimensions; it is not a model predictor

Table 1: `movieId` connects ratings, catalog records, and Genome scores; `tagId` connects each Genome score to its descriptive tag label.

Free-text user `tags` and external IMDb/TMDb identifiers from `links` are not predictors in the final model.

Evaluation follows time: training uses events before 1 January 2012; validation covers 2012 and 2013; test uses events from 1 January 2014 through the end of the extract. Each partition includes its start and excludes its end. Model parameters are fitted on training data, with validation used for early stopping. Historical feature state continues to update after each scored timestamp batch, including during validation and test. Thus later test predictions may use earlier test outcomes, but never their own or future outcomes. This evaluates a fixed model with continually refreshed history.

1995–2011	2012–2013	2014–2015
TRAIN	VALIDATION	TEST
Learn XGBoost parameters	Select the fitted model and apply early stopping	Evaluate the fixed fitted model
Historical feature state continues to evolve through time		
read events with $t_e < t \rightarrow$ score every event at $t \rightarrow$ add the complete t batch		

Figure 1: The fitted model is fixed after validation, while its point-in-time inputs use newly available prior outcomes. Earlier test outcomes may inform later test features without revealing the current or future target.

1.2 Why point-in-time construction matters

An aggregate computed over a user’s entire history would reveal later ratings to earlier predictions. The project instead defines eligible history as

$$H(t) = \{e : t_e < t\}.$$

Every behavioral feature uses only this strict-prior history. The current rating, its binary target, and all ratings at the same timestamp are excluded. Because source timestamps have one-second resolution and no known within-second ordering, the pipeline computes all feature rows at t before adding that entire batch to history. IDs identify rows; they do not determine chronology. The rules are specified in `docs/TEMPORAL_CONTRACT.md`.

The temporal guarantee has a defined scope. Timestamps are timezone-naive, and the data cannot verify when an outcome actually arrived in a live system. Catalog and Genome metadata are undated snapshots, assumed available at scoring time rather than reconstructed historically. Behavioral histories obey $t_e < t$; the available extract cannot establish that every static input existed at each historical prediction time.

2 Feature engineering: behavior, reception, and taste

The final model’s 30 predictors describe six complementary questions about the user, the movie, and their match (Table 2). Event, user, and movie IDs and the raw timestamp are alignment keys, not predictors. Features are materialized by `src/features/materialize.py`; the definitions below follow the executable 30-feature contract.

2.1 Behavioral and catalog signals

Rating style and historical reception. Global count and mean describe the accumulated rating stream. User count, mean, population standard deviation, and time since the last rating describe activity, generosity, consistency, and recency. Movie count, mean, and population standard deviation describe the amount of evidence, historical reception, and disagreement among raters.

Family	What it captures	Representative features
User behavior / rating style	Activity, generosity, dispersion, and recency	<code>user_rating_count</code> , <code>user_mean_rating</code> , <code>user_rating_std_pop</code>
Historical movie reception	Long-run audience evidence and sentiment	<code>movie_rating_count</code> , <code>movie_mean_rating</code> , <code>movie_rating_std_pop</code>
Recent movie reception	Rating activity and sentiment over 30- and 60-day windows	<code>movie_rating_avg_60d</code> , <code>movie_rating_count_30d_over_60d</code>
Genre affinity	The user’s prior response to the target movie’s genres	<code>user_target_genre_mean_rating</code> , <code>user_target_genre_mean_delta</code>
Catalog context	Breadth of genres, release year, missingness, and approximate age	<code>movie_genre_count</code> , <code>movie_release_year</code> , <code>movie_age_years</code>
Genome content	Semantic match to user history and descriptors of the target movie	<code>genome_preference_margin</code> , <code>genome_nearest_liked_similarity</code> , <code>genome_movie_top10_mean</code>

Table 2: The feature set combines rating behavior, audience reception, catalog context, and semantic content rather than relying on raw user or movie IDs.

Recent audience response. Movie rating counts and means over $[t - 30 \text{ days}, t)$ and $[t - 60 \text{ days}, t)$ distinguish recent activity from recent sentiment. The count ratio `movie_rating_count_30d_over_60d` lies in $[0, 1]$ when defined: a value above 0.5 means more ratings arrived in the latest 30 days than in the preceding 30. It measures rating activity, not viewing volume. The average-rating ratio `movie_rating_avg_30d_over_60d` is above 1 when recent reception exceeds the 60-day average; unlike the count ratio, it is not bounded above by 1.

Genre affinity. The user’s count and mean for the target movie’s genres, plus the difference from the user’s overall mean, measure experience and relative preference. Counts represent rating–genre associations: a prior movie sharing two target genres contributes twice. The genre mean uses the same association weighting, rather than a simple mean over distinct prior movies.

Catalog context. Genre count, release year, release-year missingness, and movie age supply four additional predictors. Release year is parsed from the title. Age is elapsed time from 1 January of that year, divided by 365.2425 days; it is an approximation rather than time since an exact release date. Negative ages are retained as source inconsistencies.

2.2 Genome: comparing a movie with the user’s preferences

Genre labels are broad. MovieLens Genome provides a richer description: each covered movie has a vector of relevance scores for 1,128 descriptive tags. Scores lie in $[0, 1]$, with larger values indicating a stronger association. These are content relevance scores, not the user’s ratings or probabilities of liking the movie.

Cosine similarity compares the pattern of emphasis across tags, independently of overall vector magnitude. For a target vector $\mathbf{g}_m$ and another movie or profile $\mathbf{a}$,

$$\cos(\mathbf{g}_m, \mathbf{a}) = \frac{\mathbf{g}_m \cdot \mathbf{a}}{\|\mathbf{g}_m\| \|\mathbf{a}\|}.$$

Because the relevance vectors are nonnegative, these similarities lie in $[0, 1]$, up to floating-point precision.

The user’s positive profile aggregates covered movies rated at least 4 before t ; the negative profile aggregates covered movies rated below 4 before t . Denote these histories by $L_u(t)$ and $N_u(t)$, respectively. The negative group is a target class, not necessarily an expression of dislike: it includes 3.5-star ratings. Each historical rating contributes its movie’s *raw* vector with equal

weight:

$$\mathbf{p}_u^+(t) = \sum_{j \in L_u(t)} \mathbf{g}_{m_j}, \quad \mathbf{p}_u^-(t) = \sum_{j \in N_u(t)} \mathbf{g}_{m_j}.$$

The implementation sums vectors; using their arithmetic mean gives the same cosine direction. Ratings determine group membership, not vector weights. Table 3 distinguishes the five affinity features.

Feature	Question answered
genome_user_positive_cosine	How similar is the target to the aggregate positive profile?
genome_user_negative_cosine	How similar is it to the aggregate below-four-star profile?
genome_preference_margin	Is it closer to the positive profile? Positive cosine minus negative cosine; range $[-1, 1]$.
genome_nearest_liked_similarity	How similar is it to the single closest previously liked movie?
genome_top5_liked_similarity	How similar is it, on average, to the five closest liked movies? Uses all available when fewer than five exist.

Table 3: Genome affinity captures both broad preference alignment and resemblance to specific previously liked movies; all contributing ratings precede t .

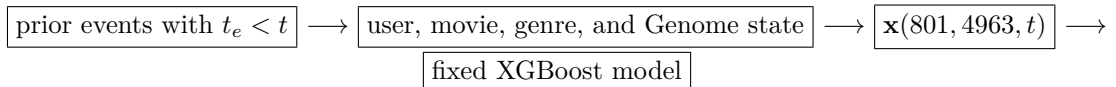
Three further Genome predictors describe the target movie alone: the mean and population standard deviation of its relevance scores, and the mean of its ten largest scores. Their definitions use the complete vector, independently of the user.

2.3 Worked prediction row: user 801 and Ocean’s Eleven

The saved training-data notebook verifies one row end to end. Immediately before 26 February 2007 at 20:28:58, user 801 had 20 earlier ratings. Eight were positive and twelve were below four stars; all 20 had valid Genome vectors. The pipeline transforms those events, earlier movie ratings, and the frozen metadata into the representative values in Table 4.

Feature	Value	Interpretation before the target rating
user_rating_count	20	Earlier ratings by user 801
user_mean_rating	3.3250	User’s historical rating level
user_rating_std_pop	1.1099	Historical rating dispersion
movie_rating_count	11,829	Earlier ratings of the target movie
movie_mean_rating	3.7716	Target movie’s historical reception
movie_rating_avg_60d	3.6771	Mean over 511 ratings in the prior 60 days
user_target_genre_mean_rating	3.2222	User’s mean for Crime/Thriller associations; 0.1028 below the user’s mean
genome_user_positive_cosine	0.784841	Similarity to the positive profile
genome_user_negative_cosine	0.865814	Similarity to the below-four-star profile
genome_preference_margin	-0.080973	Positive minus negative cosine

Table 4: A point-in-time subset of the 30-feature vector for rating event 119139. Every behavioral value is computed before the event timestamp.



The two Genome cosines give

$$\text{preference margin} = 0.784841 - 0.865814 = -0.080973,$$

so the target is closer to the user’s below-four-star profile. Only after the feature vector is constructed is the outcome observed: the rating is 2.5, so $y = 0$. The observed rating is a label, not an input. The notebook does not save a predicted probability for this row, so none is reported here. The example is reproduced in `notebooks/model/genome_prd_v1/01_training_dataset.ipynb`.

2.4 Ranges and missing history

Historical rating means remain on the $[0.5, 5.0]$ rating scale; population standard deviations lie in $[0, 2.25]$. These are definition-based bounds, not claims about observed minima and maxima. Counts are nonnegative. Empty user and movie histories use the global mean available at t , which starts at the fixed prior 3.5. Empty or singleton histories have zero population standard deviation. With no matching genre history, the genre mean falls back to the user mean and the genre delta is zero.

Empty rolling windows have zero counts and missing means. The count ratio is missing when the 60-day count is zero; the mean ratio is missing when either window mean is unavailable. Recency and unavailable release metadata remain missing. Genome profiles ignore historical movies without complete, nonzero vectors. Affinity values are missing if their required target or profile is unavailable; the margin requires both profiles. A movie with a valid vector still has static Genome statistics even when the user has no history. XGBoost handles the remaining missing values natively.

3 Modeling and temporal results

XGBoost combines the feature families through nonlinear tree decisions, allowing the relevance of a movie’s reception to depend on the user’s history and taste. The fitted model contains 600 trees of maximum depth 6, with learning rate 0.05. Validation early stopping was configured with patience 30; the best iteration was 599 (zero-based), so this run reached the 600-tree limit. Training is recorded in `notebooks/model/genome_prd_v1/03_xgboost_training.ipynb`; `04_model_evaluation.ipynb` in the same directory produces the results.

3.1 Ranking generalizes to later events

ROC-AUC measures separation between positive and negative events across score thresholds. Average Precision (AP) summarizes precision as recall increases; its interpretation depends on the prevalence of positive outcomes. The repository’s metric named PR-AUC is computed with `average_precision_score`, so this report calls it AP rather than implying trapezoidal integration of a precision–recall curve.

Split	Events	Positive rate	AP	ROC-AUC	Log loss	Brier
Train	17,822,773	49.8%	0.8111	0.8221	0.5134	0.1719
Validation	1,330,716	52.0%	0.8229	0.8200	0.5159	0.1727
Test	846,774	50.2%	0.8111	0.8213	0.5162	0.1728

Table 5: Closely aligned ranking and probability-error metrics suggest limited overfitting across these temporal partitions. All metrics use the full split; AP denotes Average Precision.

On the 846,774 test events, AP is 0.8111 and ROC-AUC is 0.8213, with a high-rating prevalence of 50.2%. The small train–test gaps are consistent with limited overfitting on these observed periods, rather than proof of future stability. Across the five test quarters, ROC-AUC ranges from 0.8165 to 0.8253 and AP from 0.7955 to 0.8287. Prevalence also varies, so AP differences

are not a pure measure of model drift. These results concern later rating events; they do not isolate performance on unseen users or movies.

3.2 What Genome contributed

A saved controlled experiment compared the original 17-feature model with a 25-feature version that added the eight Genome predictors under the same training and evaluation protocol. Test AP increased from 0.7799 to 0.8099 and ROC-AUC from 0.7953 to 0.8204. AP, ROC-AUC, log loss, and Brier score improved in all five test quarters. This provides evidence for the Genome family as a whole; it does not isolate each Genome feature’s incremental value.

That experiment predates the current 30-feature model. Its improvement must not be attributed to the later rolling-window additions. No controlled ablation establishes their incremental benefit in the reviewed evidence. In repository terms, the feature set evolved from the 17-feature baseline to the 25-feature Genome experiment and then to the current 30-feature model. The six rolling-window definitions contribute five new columns because the 30-day movie rating count already belonged to the baseline.

3.3 From ranking to useful scores

The test events divide into 20 approximately equal-sized score groups (ventiles). Observed high-rating prevalence rises from 1.7% in the lowest group to 94.4% in the highest, without an adjacent decline (Figure 2). This is strong separation among the observed events, not measured recommendation or engagement lift.

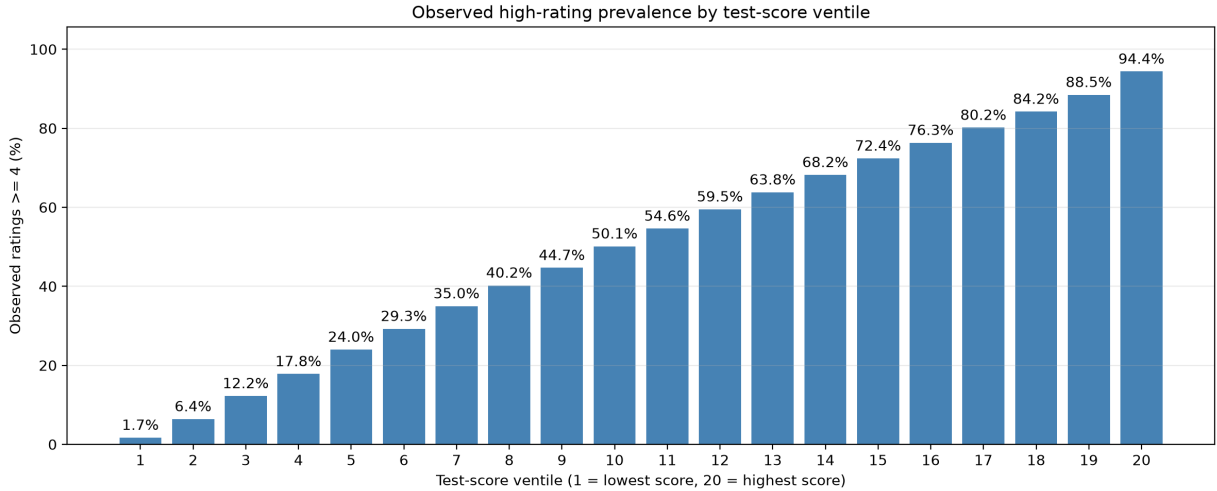


Figure 2: Higher scores consistently identify more high ratings: observed prevalence rises from 1.7% to 94.4% across the 20 test-score groups, each containing approximately 5% of test events.

Good ranking does not guarantee accurate probabilities. In all ten equal-width test probability bins, the mean prediction exceeds observed prevalence (Figure 3). The sample-weighted mean absolute bin gap, or expected calibration error, is 0.02075: about 2.1 percentage points. Test log loss is 0.5162 and Brier score is 0.1728; lower values are better for both probability-error measures.

The evaluation notebook also explores 20 test-derived cutoffs. The highest reported F1 among them is 0.7615 at a cutoff of 0.391923, selecting about 65.0% of events with precision 0.6747 and recall 0.8740. This is a descriptive test-set comparison, not an independently validated operating policy. The saved model specifies no decision threshold. A deployment cutoff would need selection against a defined objective on validation data and evaluation on fresh holdout data.

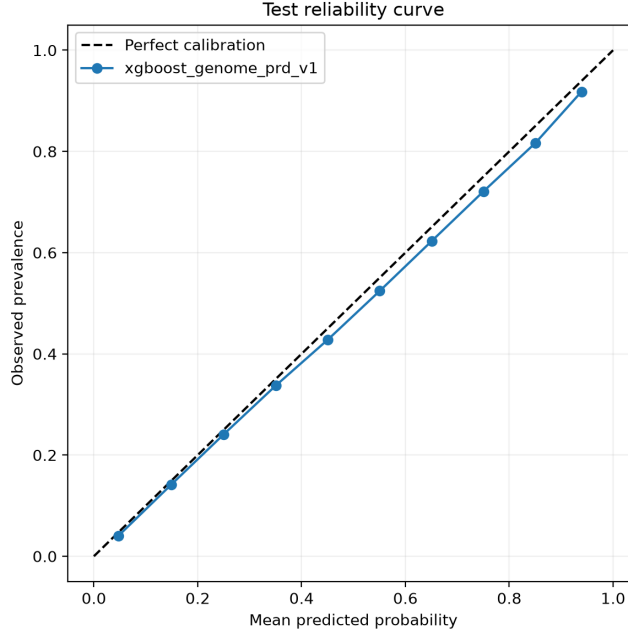


Figure 3: Predicted probabilities modestly exceed observed high-rating rates across all ten test bins. The weighted calibration gap is 2.1 percentage points; strong ranking alone does not make the scores perfectly calibrated.

4 Interpretation: what drives the predictions

SHAP explains how features move a fitted model’s prediction away from its reference value. The analysis uses 50,000 test events sampled without replacement with seed 42. Contributions add on the raw log-odds scale, before conversion to probability; they are not percentage-point effects. The saved notebook verifies that the base value plus all 30 contributions reconstructs the model’s raw output within 10^{-4} .

Predictor	Mean absolute SHAP
<code>genome_preference_margin</code>	0.5890
<code>user_mean_rating</code>	0.5248
<code>movie_mean_rating</code>	0.2012
<code>movie_rating_avg_60d</code>	0.1744
<code>user_rating_count</code>	0.1661

Table 6: Personalized taste alignment and user rating style lead the five largest mean absolute SHAP contributions, measured in log-odds on 50,000 test events. These are attribution magnitudes, not incremental model lift.

Personalized fit comes first. A higher `genome_preference_margin` generally pushes the prediction upward: the target resembles the user’s positive history more than the below-four-star history. This is the largest individual mean absolute SHAP contribution, followed by `user_mean_rating`, the user’s tendency to rate generously or strictly.

Audience reception adds context. Higher `movie_mean_rating` and `movie_rating_avg_60d` generally support higher predictions. The former describes accumulated reception; the latter captures recent sentiment. For `user_rating_count`, higher activity often has a negative contribution in this sample. That pattern describes how the model uses activity alongside other inputs; it does not show that rating more movies causes someone to like them less.

Mean absolute SHAP ranks the magnitude of model contributions, not causal importance or performance gain from adding a feature. Correlated predictors can share attribution, and the results apply to this model and test sample. They complement the controlled Genome comparison rather than replace it. The analysis and [Figure 4](#) are produced in `notebooks/model/genome_prd_v1/04_1_shap_model_interpretation.ipynb`.

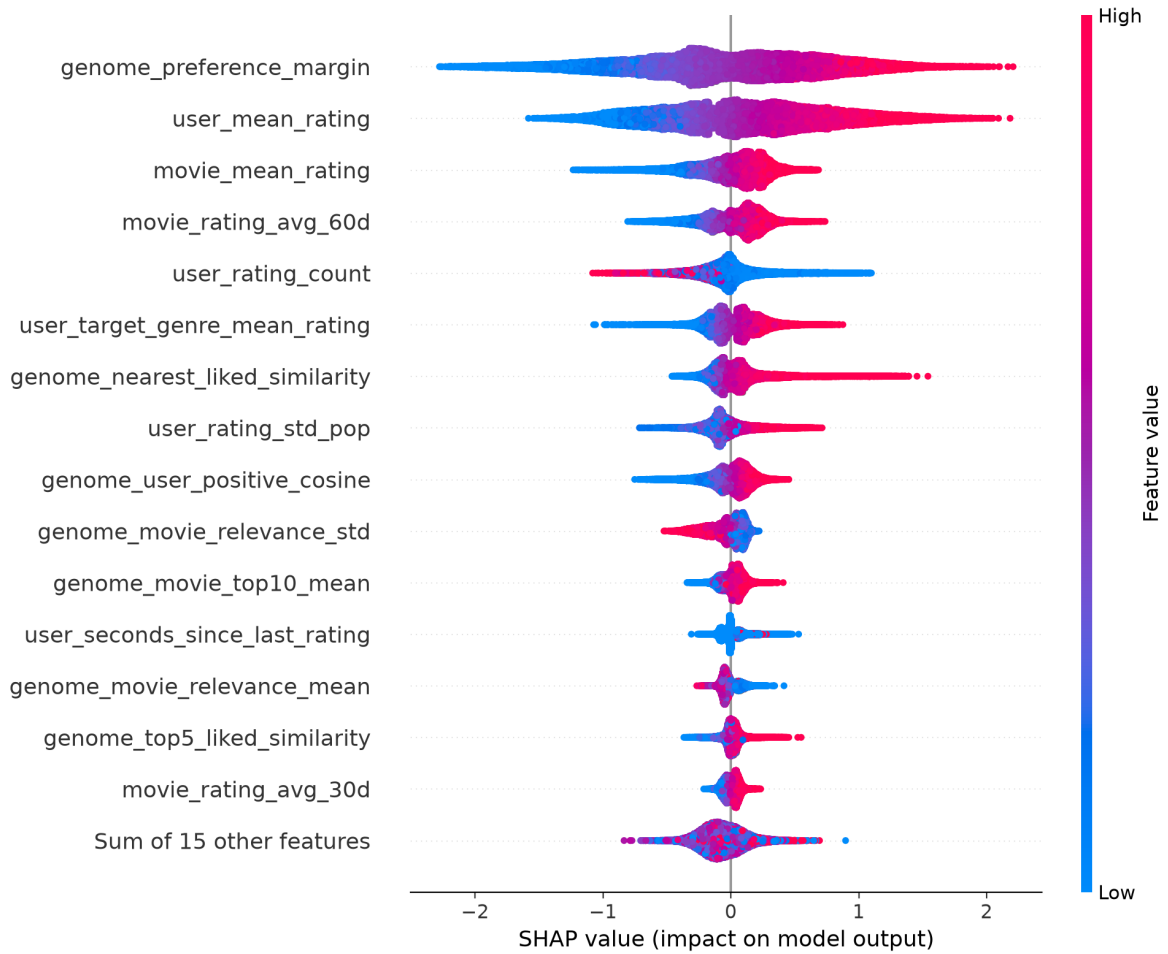


Figure 4: Taste alignment and rating style dominate individual attributions. Each point is a sampled test event: rightward contributions raise log-odds; leftward contributions lower them. Red and blue denote higher and lower feature values. The chart shows 15 individual features and the sum of the remaining 15.

5 Key takeaways

This project predicts whether an observed movie-rating event will receive at least four stars. A 30-feature XGBoost model combines user behavior, historical and recent movie reception, genre affinity, and Genome content similarity. Strict-prior feature construction makes the evaluation meaningful by preventing current, simultaneous, and future rating outcomes from entering a prediction.

The main lesson is that personalized taste alignment complements both the user’s rating style and the movie’s reception. These are the strongest SHAP signals, and the earlier controlled experiment supports the predictive value of the Genome family. The current model generalizes consistently across the observed temporal partitions, reaching test AP of 0.8111 and ROC-AUC of 0.8213.

The remaining work is specific: establish historical metadata availability where possible, reconcile the deployment manifest, and validate calibration and a decision policy before live use. The evidence supports a useful offline baseline; it does not yet establish causal effects, incremental benefit from the latest rolling features, or online business impact.